



Module 2. Supervised Learning and Experimental Design

Performance Metrics for Classification Tasks

Confusion Matrix

Precision-Recall & Fall-Out

Finding New Threshold

ROC Curves

Introduction

- A key step in machine learning algorithm development and testing is determining a good error and evaluation metric.
- Evaluation metrics help us to estimate how well our model is trained and it is important to pick a metric that matches our overall goal for the system.
- Some common evaluation metrics include **precision, recall, receiver operating curves, and confusion matrices.**

A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps, extending from the top to the bottom.

Confusion Matrix

Confusion Matrix

- A confusion matrix summarizes the classification accuracy across several classes.
- It shows the ways in which the classification model is **confused** when it makes predictions, allowing visualization of the performance of our algorithm.
- Generally, each row represents the instances of a actual class while each column represents the instances in an predicted class.
- **All correct predictions are located in the diagonal of the table.** So it is easy to visually inspect the table for prediction errors, as they will be represented by values outside the diagonal.
- The confusion matrix for this two-class classification problem is:

actual/predict	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Confusion Matrix

- **True positive (TP):** correctly predicting positive events
- **False positive (FP):** incorrectly calling positive to a negative event
- **True negative (TN):** correctly predicting negative events
- **False negative (FN):** incorrectly labeling negative to a positive event

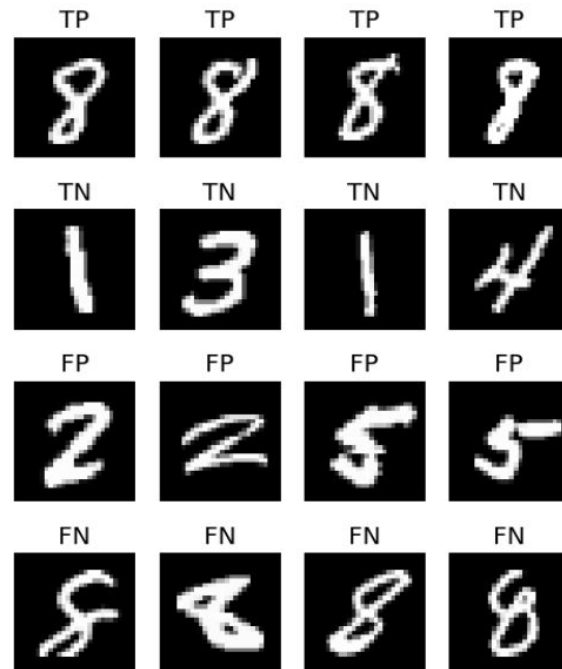
actual/predict	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Confusion Matrix - Examples of Assignments

```
idx_tp = np.where((t_train_8==1)&(y_train==1))[0]
idx_tn = np.where((t_train_8==0)&(y_train==0))[0]
idx_fp = np.where((t_train_8==0)&(y_train==1))[0]
idx_fn = np.where((t_train_8==1)&(y_train==0))[0]

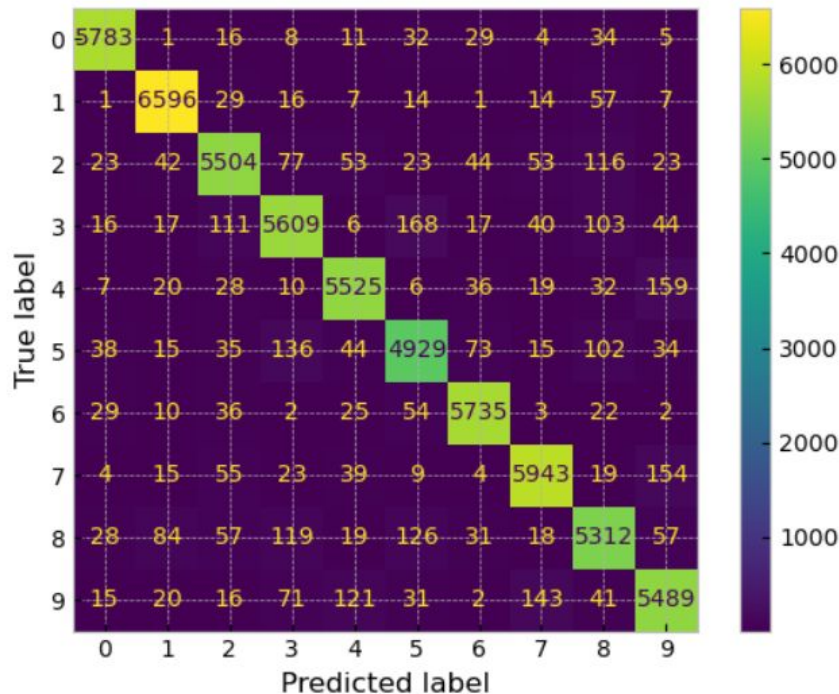
idx = np.vstack((idx_tp[:10], idx_tn[:10], idx_fp[:10], idx_fn[:10]))

plt.figure(figsize=(15,7))
labels=['TP', 'TN', 'FP', 'FN']
grid=1
for i in range(4):
    for j in range(10):
        plt.subplot(4,10,grid)
        plt.imshow(X_train[idx[i,j],:].reshape(28,28), cmap='gray')
        plt.axis('off')
        plt.title(labels[i])
        grid+=1
```



Confusion Matrix - Visualization

```
1 from sklearn.metrics import ConfusionMatrixDisplay
2
3 disp = ConfusionMatrixDisplay(cm, display_labels=range(10));
4 disp.plot();
```



Confusion Matrix - Focus on errors

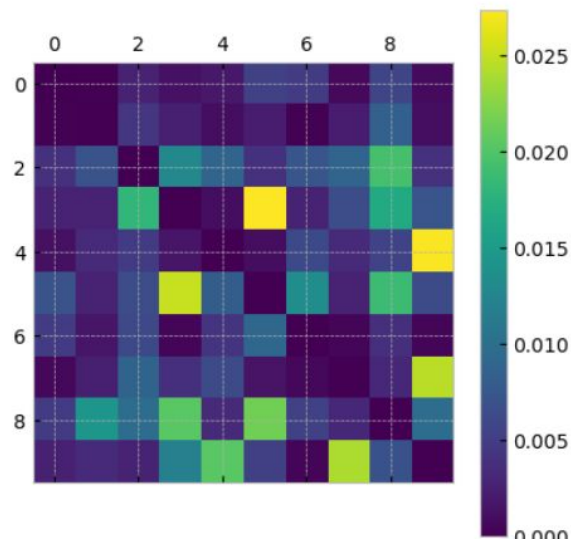
Start by normalizing by the sample size per class:

```
In [33]: 1 row_sums = cm.sum(axis=1, keepdims=True)
          2
          3 norm_conf_mx = cm / row_sums
```

Next, fill the diagonal with zeros:

```
In [34]: 1 np.fill_diagonal(norm_conf_mx, 0)
```

```
In [35]: 1 plt.matshow(norm_conf_mx)
          2 plt.colorbar();
```



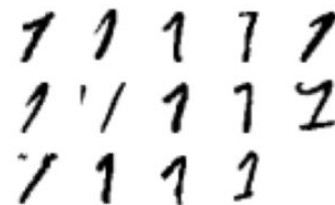
Confusion Matrix - Focus on errors

The code for plotting this is on
its respective Jupyter
Notebook in Github

True Positives



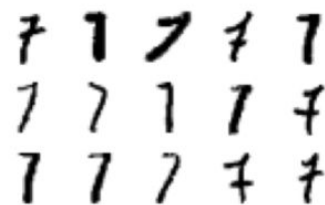
False Negatives



True Negatives



False Positives



A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps, extending from the top to the bottom.

Precision-Recall & Fall-Out

Precision

- **Precision**, also called Positive Predictive Value (PPV), is the performance of detection

$$\text{Precision} = \text{PPV} = \frac{TP}{TP + FP}$$

```
1 precision_score(t_train_8, y_train)
np.float64(0.8643410852713178)
```

Recall

- **Recall**, also called True Positive Rate (TPR) or Sensitivity, is the probability of detection

$$\text{Recall} = \text{TPR} = \text{Sensitivity} = \frac{TP}{TP + FN}$$

```
1 recall_score(t_train_8, y_train)
np.float64(0.7241497179969236)
```

F1-score

- **F1-score**, also called F-score or F-measure, is a measure of a model's accuracy. It considers both the precision and the recall.
- The F1 score favors classifiers that have similar precision and recall. Unfortunately, you can't have it both ways: increasing precision reduces recall, and vice versa. This is called the precision/recall trade-off.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

```
1 f1_score(t_train_8, y_train)
np.float64(0.7880591462847577)
```

- **Fall-out**, also called False Positive Rate (FPR), is the probability of false alarm
- **Specificity**, also called True Negative Rate (TNR), is the probability of negative events detection

$$\text{Fall-out} = \text{FPR} = \frac{FP}{FP + TN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}$$



Finding New Threshold

New Threshold in Binary Classification

- In this binary classifier, the decision surface is as follows:

$$\begin{cases} 1 & \mathbf{w}^T \mathbf{x} + w_0 \geq 0 \\ 0 & \mathbf{w}^T \mathbf{x} + w_0 < 0 \end{cases}$$

- The threshold $\delta=0$ can be changed to maximize some performance measure: For instance finding threshold where precision = 90 degrees

$$\begin{cases} 1 & y(\mathbf{x}) = \phi(\mathbf{w}^T \mathbf{x} + w_0) \geq \delta \\ 0 & y(\mathbf{x}) = \phi(\mathbf{w}^T \mathbf{x} + w_0) < \delta \end{cases}$$

New Threshold in Binary Classification

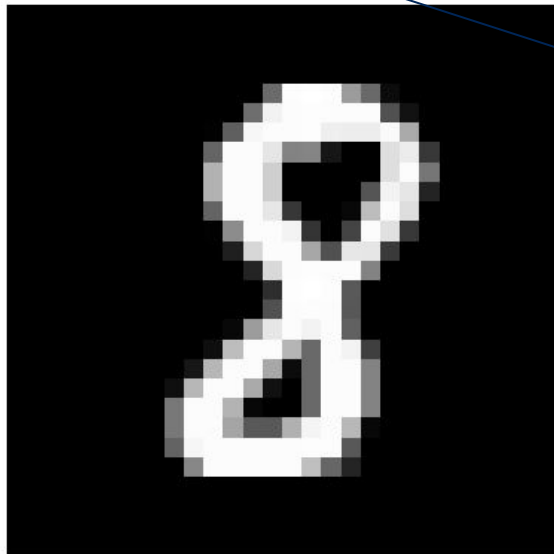
```

1 idx=24_000
2 plt.imshow(X_train[idx].reshape(28,28), cmap='gray')
3 plt.title(t_train[idx]); plt.axis('off');
4
5 y_scores = model.decision_function([X_train[idx]]) # computes the regression part, w^T x + w_0
6
7 y_scores

```

array([4.13871708])

8



$$\begin{cases} 1 & \mathbf{w}^T \mathbf{x} + w_0 \geq 0 \\ 0 & \mathbf{w}^T \mathbf{x} + w_0 < 0 \end{cases}$$

New Threshold in Binary Classification

```
In [13]: 1 1/(1+np.exp(-y_scores))
```

```
Out[13]: array([0.98430691])
```

```
In [14]: 1 model.predict_proba([X_train[idx]]), model.predict([X_train[idx]])
        2 # uses threshold of 0.5 by default
```

```
Out[14]: (array([[0.01569309, 0.98430691]]), array([1]))
```

```
In [15]: 1 threshold = 0 # for regression part,  $w^T X + w_0$ 
        2
        3 y_some_digit_pred = y_scores > threshold
        4 y_some_digit_pred
```

```
Out[15]: array([ True])
```

```
In [16]: 1 threshold = 8000
        2
        3 y_some_digit_pred = y_scores > threshold
        4
        5 y_some_digit_pred
```

```
Out[16]: array([False])
```

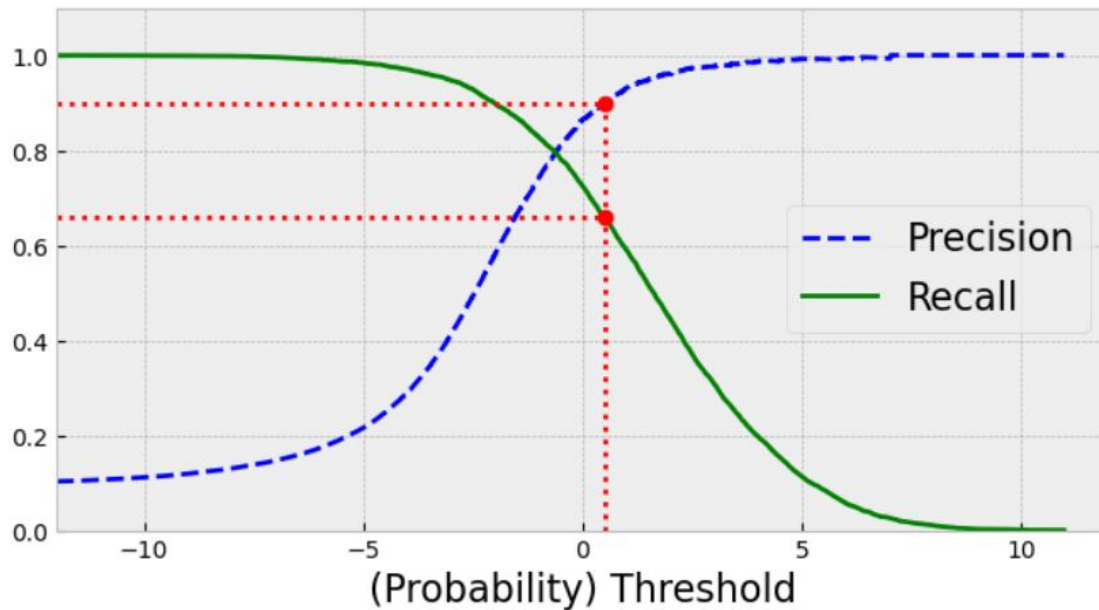
```
In [17]: 1 y_scores = model.decision_function(X_train)
        2
        3 y_scores
```

```
Out[17]: array([-10.69140914, -8.28294243, -11.46589105, ..., -3.75523349,
               -14.34637969,  0.14322034])
```

Finding Threshold where Precision $\geq 90\%$

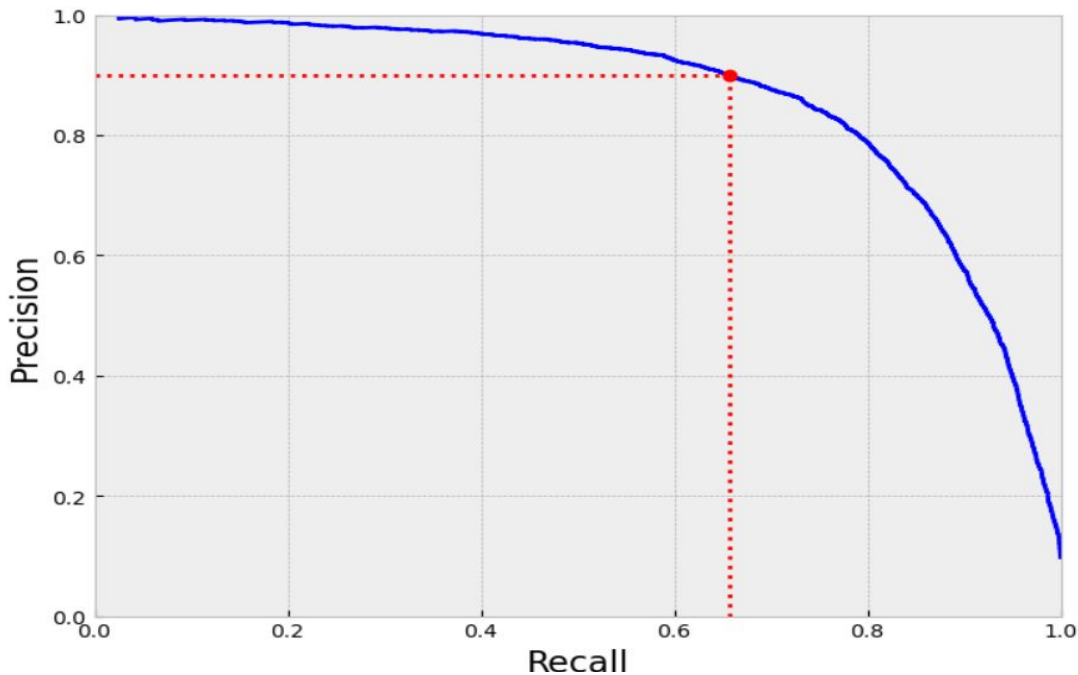
```
from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds = precision_recall_curve(t_train_8, y_scores)
```

The code for plotting this is on its respective Jupyter Notebook in Github



Finding Threshold where Precision $\geq 90\%$

Another way to select a good precision/recall trade-off is to plot precision directly against recall.



Finding Threshold where Precision $\geq 90\%$

```
1 threshold_90_precision = thresholds[np.argmax(precisions >= 0.90)]
2 y_train_pred_90 = 1*(y_scores>=threshold_90_precision)
```

```
1 precision_score(t_train_8, y_train_pred_90)
```

```
np.float64(0.9000935891436593)
```

```
1 recall_score(t_train_8, y_train_pred_90)
```

```
np.float64(0.6574944453939497)
```

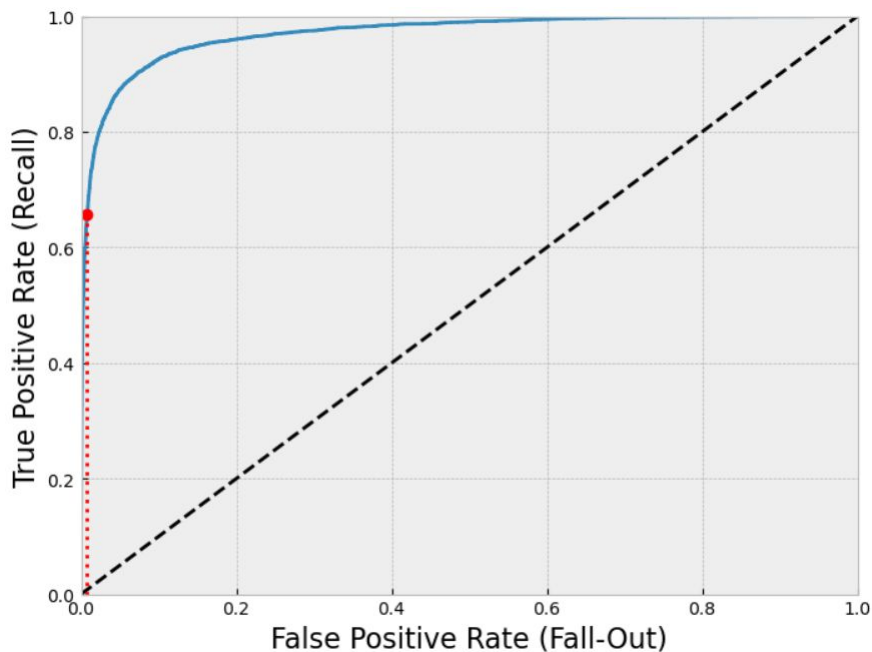
ROC Curves

ROC Curves

- **Receiver Operating Characteristic (ROC)** curve is the plot between the true positive rate (TPR) and the false positive rate (FPR), where the TPR is defined as the y-axis and FPR is defined as the x-axis.
 - ROC curves were first developed for RADAR systems, hence the name.
 - Given a binary classifier and its threshold, the (x,y) coordinates of ROC space can be calculated from all the prediction result. You trace out a ROC curve by varying the threshold to get all of the points on the ROC.
 - The diagonal between (0,0) and (1,1) separates the ROC space into two areas, which are left up area and right bottom area. The points above the diagonal represent good classification (better than random guess) which below the diagonal represent bad classification (worse than random guess).
 - **What is the perfect prediction point in a ROC curve?**

ROC Curves

```
1 from sklearn.metrics import roc_curve
2
3 fpr, tpr, thresholds = roc_curve(t_train_8, y_scores)
```



Area Under the Curve

- One way to compare classifiers is to measure the area under the curve (AUC). AUC is simply the area under the ROC curve.
- Random guessing can achieve the diagonal line, so the minimum AUC is 1/2.
- The maximum AUC is 1, which is achieved by a test that is always right; the ROC curve is along the left and top axes.

```
1 from sklearn.metrics import roc_auc_score
2
3 roc_auc_score(t_train_8, y_scores)
np.float64(0.9701866829348704)
```



Project 1

Project 1

Project 1

```
1 songs = pd.read_csv('datasets\Spotify_Song_Attributes.csv')
2
3 songs.head(5)
```

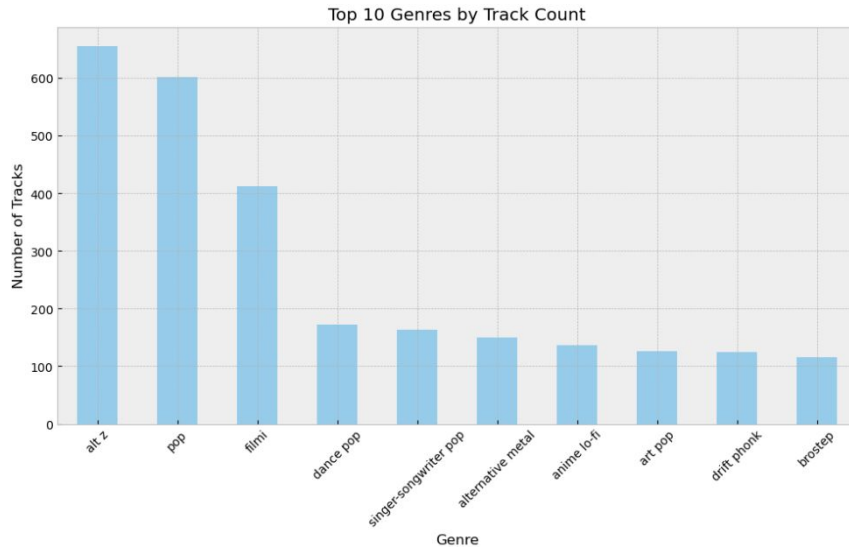
	trackName	artistName	msPlayed	genre	danceability	energy	key	loudness	mode	speechiness	...	liveness	valence	tempo
0	"Honest"	Nico Collins	191772	NaN	0.476	0.799	4.0	-4.939	0.0	0.2120	...	0.2570	0.577	162.139
1	"In The Hall Of The Mountain King" from Peer G...	London Symphony Orchestra	1806234	british orchestra	0.475	0.130	7.0	-17.719	1.0	0.0510	...	0.1010	0.122	112.241
2	#BrooklynBloodPop!	SyKo	145610	glitchcore	0.691	0.814	1.0	-3.788	0.0	0.1170	...	0.3660	0.509	132.012
3	\$10	Good Morning	25058	experimental pop	0.624	0.596	4.0	-9.804	1.0	0.0314	...	0.1190	0.896	120.969
4	(I Just) Died In Your Arms	Cutting Crew	5504949	album rock	0.625	0.726	11.0	-11.402	0.0	0.0444	...	0.0625	0.507	124.945

5 rows × 22 columns

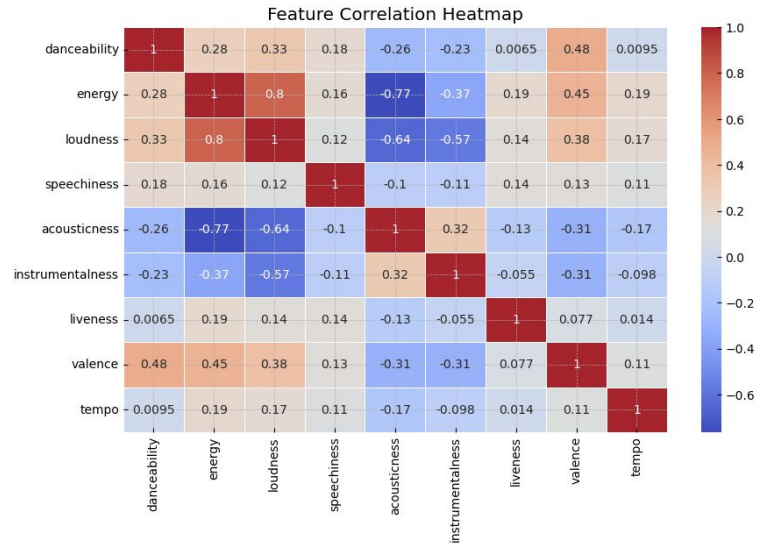
Project 1

In [44]:

```
1 # Top Genres by Track Count
2 if 'genre' in songs.columns:
3     plt.figure(figsize=(12, 6))
4     songs['genre'].value_counts().nlargest(10).plot(kind='bar', color='skyblue')
5     plt.xlabel("Genre")
6     plt.ylabel("Number of Tracks")
7     plt.title("Top 10 Genres by Track Count")
8     plt.xticks(rotation=45)
9     plt.show()
```



```
1 # Correlation Heatmap
2 plt.figure(figsize=(10, 6))
3 corr_matrix = songs[['danceability', 'energy', 'loudness', 'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo']]
4 sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", linewidths=0.5)
5 plt.title("Feature Correlation Heatmap")
6 plt.show()
7
```



Project 1

```
1 # Top Artists by Track Count
2 plt.figure(figsize=(12, 6))
3 top_artists = songs['artistName'].value_counts().nlargest(10)
4 sns.barplot(x=top_artists.index, y=top_artists.values, palette="viridis")
5 plt.xlabel("Artist")
6 plt.ylabel("Number of Songs")
7 plt.title("Top 10 Artists with Most Tracks")
8 plt.xticks(rotation=45)
9 plt.show()
```

C:\Users\aramirezsalgado\AppData\Local\Temp\ipykernel_31932\3551642860.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=top_artists.index, y=top_artists.values, palette="viridis")
```

